

# Hexagon

---

for TypeScript coders

---

**A note before we begin.** Hexagon is a language currently being designed. The compiler, `hexc`, is under active development against a formal specification, but there is nothing you can `npm install` today. What follows is a tour of a language as specified — every example in this book is drawn from the spec documents and shows the language as it will behave. Consider this a preview, written for the curious.

---

## Who this book is for

You. You've written serious TypeScript. You know why `const` is better than `let`, you've fought `strictNullChecks` battles and won, you can write a mapped type when you have to (and you'd rather not have to). You've heard the phrase "functional programming" and filed it under *probably interesting, probably involves monads, no time right now*.

Good news: this book contains no monads. Hexagon is a functional language, but it's built for people exactly like you — it compiles to JavaScript you could have written by hand, ships honest `.d.ts` files, and borrows its surface feel from the pragmatic end of the ML family (F#, OCaml, Roc) rather than the academic end.

This book is small and incomplete on purpose. It shows off the parts most likely to make a TypeScript developer sit up. We'll go feature by feature, and wherever possible we'll do it side by side: the TypeScript you'd write today on the left of your mind, the Hexagon on the right.

---

## Chapter 1 — The whole language in one function

Here is a complete Hexagon function:

```
let sumTo(n) =  
  var total = 0  
  for i in 1..n  
    total := total + i  
  total
```

And here is the JavaScript the compiler emits for it:

```
const sumTo = (n) => {  
  let total = 0;  
  for (let i = 1; i <= n; i++) {  
    total = total + i;  
  }  
}
```

```
    return total;
};
```

Two things to notice, and they set the theme for the whole book.

**First: there are no type annotations, yet this function is fully, strongly typed.** The compiler inferred `sumTo : (Int) -> Int` — not guessed, not defaulted to *any*, *proved*. If you call `sumTo("ten")` you get a compile error. We'll spend all of Chapter 2 on this, because it's Hexagon's headline feature.

**Second: the output is the JavaScript you would have written.** No framework runtime bolted on, no wrapper objects, no mangled names — at most a few small, readable helpers where a checked operation needs one. Readable output is a *semantic constraint* in Hexagon's design — features that couldn't compile to clean JS were rejected during design, not patched over afterward. When you debug, you debug code you can read. When a teammate who doesn't know Hexagon opens the output, they see ordinary JavaScript with an ordinary `.d.ts` next to it.

That's the pitch. Now the details.

---

## Chapter 2 — Strong type inference, zero annotations

This is interesting feature number one.

### 2.1 The TypeScript situation

TypeScript's inference is good — genuinely good — but you know where it stops. It infers *outward* from values, but function parameters are a wall:

```
// TypeScript
function firstWord(s) {           // s: any - inference gives up at the
  parameter
  return s.split(" ")[0];
}

function firstWord(s: string): string { // so you annotate
  return s.split(" ")[0];
}
```

And the moment you go generic, you're writing the types yourself:

```
// TypeScript
function pair<A, B>(x: A, y: B): [A, B] {
  return [x, y];
}
```

You declare `<A, B>`, you thread them through, you maintain them. TypeScript checks your homework; it doesn't do it.

## 2.2 The Hexagon situation

Hexagon uses Hindley–Milner type inference — the ML-family algorithm that TypeScript's designers deliberately traded away for JS compatibility. In Hexagon:

```
let firstWord(s) = split(s, " ")[1]
// inferred: firstWord : (String) -> String

let pair(x, y) = (x, y) // tuple
// inferred: pair : (a, b) -> (a, b)  - generic over both, automatically
```

No annotation on `s`. No `<A, B>` declaration. The compiler works out that `pair` is generic in both arguments and *generalizes* it for you — the lowercase `a` and `b` in the inferred type are type variables, Hexagon's equivalent of `<A, B>`, except you never have to write them.

This is not "the compiler defaults to `any` and hopes." Every expression in a Hexagon program has a precise type, all the time. There is no `any`, no `as`, no `!`, no `@ts-ignore`. If the program compiles, the types hold.

## 2.3 It even infers structural requirements

Here's the one that tends to convert people. Write a function that accesses a field:

```
let getX(r) = r.x
```

What's the type of `r`? TypeScript would need `r: { x: number }` or a generic with a constraint. Hexagon *infers* the requirement:

```
getX({x: 1.0, y: 2.0})    // fine: has an x
getX({x: 1.0})           // fine: has an x
getX({y: 2.0})           // compile error: no field x
```

The inferred type says, in effect, "any record that has at least a field `x`" — and the compiler figured that out from the body alone. (The machinery behind this is called row polymorphism; you will never need to know that, and Hexagon's error messages are forbidden from using the word.)

## 2.4 Numbers are inferred too

```
let double(x) = x + x
```

Should `double` work on `Int`? `Float`? Both? Hexagon infers that `double` works for *any numeric type* — the inferred signature carries a constraint, and if you ever wanted to write it explicitly, it looks like this:

```
let double<a: Num>(x: a): a = x + x
```

`<a: Num>` reads as "for any type `a` that is numeric." It's the moral equivalent of a TS generic with a constraint (`<T extends Numeric>`), except that (a) Hexagon infers it, and (b) it's checked soundly — there's no structural loophole.

## 2.5 So when *do* you write types?

When you want to. Annotations are always legal and always checked:

```
let area(w: Float, h: Float): Float = w * h
```

Idiomatic Hexagon annotates public API signatures for documentation and lets inference run everything internal. Your editor shows the inferred type on hover either way. The difference from TypeScript is philosophical: in TS, annotations are load-bearing — remove them and safety degrades toward `any`. In Hexagon, annotations are documentation — remove them all and the program is exactly as safe as before.

---

## Chapter 3 — The basics, quickly

A short chapter to make the rest of the examples readable.

### 3.1 Bindings: `let` does everything

```
let name = "Ada"           // immutable binding (your beloved const)
let year = 1815

let greet(who) =           // functions too: this is header sugar for
  "Hello, ${who}!"         // let greet = (who) => "Hello, ${who}!"
```

`let` is `const`: immutable, block-scoped, and it binds *everything* — numbers, strings, records, and functions alike, because a function is just another value. The header form `let greet(who) = ...` and the lambda form `let greet = (who) => ...` are the same declaration; use whichever reads better.

One deliberate limitation: `let` is **non-recursive**. Inside the right-hand side, the name being bound doesn't exist yet, so `let f = n => f(n - 1)` is a compile error. For the occasional function that must call itself, there's a second keyword, `fun`:

```
fun fib(n) =
  if n < 2 then n else fib(n - 1) + fib(n - 2)
```

`fun` is hoisted (like a JS `function` declaration) and supports self- and mutual recursion. That's its entire job — it's the special case, not the default. Everyday Hexagon is `let` all the way down, and you'll notice

every other function in this book is one; recursion is rarer than you'd think in a language with `for`, `while`, and a pipeline-friendly `stdlib`.

Note the string: `"Hello, ${who}!"` — interpolation works the way your fingers already type it.

### 3.2 Everything is an expression

There are no statements in Hexagon. `if` is an expression (there is no `?:` ternary because `if` is the ternary):

```
let label = if n == 1 then "item" else "items"
```

Blocks are indentation-based (think Python, or the F#/Haskell layout tradition), and the last expression in a block is its value:

```
let describe(n) =
  let sign = if n < 0 then "negative" else "non-negative"
  let parity = if isEven(n) then "even" else "odd"
  "${n} is ${sign} and ${parity}"    // this is the return value; no
  `return` keyword
```

Braces are never blocks in Hexagon — braces always mean records (Chapter 8).

### 3.3 Operators: words, not symbols

```
if not done and count > 0 or force
  retry()
```

Logic is spelled `not`, `and`, `or` (plus `implies` and `iff` for the mathematically inclined). Writing `!`, `&&`, or `||` is a lexer error with a fix-it pointing you to the word. `and/or` short-circuit and compile to JS `&&/||`.

Two more differences from JS that remove entire bug classes:

**No truthiness.** Conditions must be `Bool`. `if count` where `count : Int` is a type error, full stop — the compiler will not suggest a coercion because there isn't one.

**Comparison chaining.** This works, and means what it says:

```
if 0 <= x and x <= 10 ...    // fine, but also:
if 0 <= x <= 10 ...          // legal Hexagon: chained comparison, math
semantics
```

In JS, `0 <= x <= 10` silently evaluates to nonsense via boolean coercion. Hexagon gives it the meaning your math teacher intended.

## Strings concatenate with `++`, not `+`.

```
let full = first ++ " " ++ last
```

`+` is arithmetic only. Write `"a" + "b"` and the error says: *did you mean ++*? The upside is that JS's mixed-operand `+` accidents (`1 + "2"`) are unrepresentable.

## 3.4 Numbers

`Int` and `Float` are distinct types (finally!), and both compile to the JS `number` you already have — `Int` is a compile-time discipline guaranteeing whole-number arithmetic, not a boxed runtime type. `1` is an integer literal, `1.0` is a float literal, and there is no implicit conversion between them; you cross explicitly with `Float.fromInt(n)`. Integer division by zero throws `DivideByZeroError` at runtime; float division follows IEEE 754 like JS does.

## 3.5 Equality that actually works

`==` in Hexagon is *structural* equality, and it's defined for tuples, records, and unions automatically:

```
{x: 1, y: 2} == {x: 1, y: 2}    // true
(1, "a") == (1, "a")           // true
Some(3) == Some(3)             // true
```

Every TypeScript developer has written a `deepEqual` helper or reached for `lodash` because `{a: 1} === {a: 1}` is `false`. In Hexagon, `==` on compound data compares contents, the compiler generates the comparison code, and it's a *type error* to `==` two things of different types. (On primitives it compiles straight to `===`.)

# Chapter 4 — Pipes

Interesting feature number two, and the one you'll miss most when you go back.

## 4.1 The problem pipes solve

You already write pipelines in TypeScript — you just call them method chains:

```
// TypeScript
const result = xs
  .map(x => x + 1)
  .filter(p)
  .slice(0, 3);
```

Chains are lovely until you need a function that *isn't* a method on the object. Then the pipeline breaks and reading order reverses:

```
// TypeScript – mixing methods and free functions
const result = take(unique(xs.map(x => x + 1).filter(p)), 3);
//          ^^^^ read me last      ^^^^^^ read me... third?
```

You read the *innermost* call first and spiral outward. Everyone has written this; nobody enjoys reading it.

## 4.2 The pipe operator

Hexagon's `|>` takes the value on its left and inserts it as the **first argument** of the call on its right:

```
xs |> map(x => x + 1) |> filter(p) |> take(3)
```

This is defined as a purely mechanical rewrite:

```
a |> f(b, c)      // becomes  f(a, b, c)
x |> negate      // becomes  negate(x)      (bare form: no parens needed)
```

So the pipeline above is `take(filter(map(xs, x => x + 1), p), 3)` — the same nested call you'd have written, but spelled in reading order: start with `xs`, add one to each, filter, take three. Data flows left to right, the way the steps happen.

Because it's just a rewrite, pipes work with **every** function — `stdlib`, yours, anyone's. No waiting for the standard library to add a method, no prototype patching, no `pipe()` utility functions with eleven overload signatures (if you've used `fp-ts` or `Ramda`'s `pipe`, you know exactly which eleven overloads I mean).

## 4.3 The convention that makes it sing

For first-argument insertion to compose, everything must agree on where the subject goes. Hexagon's standard library has one iron rule: **the first parameter is always the thing being operated on**.

`map(list, f)`, `split(string, sep)`, `take(seq, n)`. It reads like a method call with the dot removed — `map(xs, f)` is `xs.map(f)` reshuffled — so your OO instincts transfer directly.

## 4.4 What it compiles to

Nothing. Pipes are erased before the type checker even runs; the emitted JavaScript is the plain nested calls (with intermediate `consts` lifted when nesting gets deep, for readability). Zero runtime cost, zero runtime existence.

---

## Chapter 5 — Shadowing: one simple rule

Every language has to answer: what happens when you reuse a name? JavaScript answered with a shrug (`var` hoisting, `let` redeclaration errors, parameter shadowing allowed, TDZ...). Hexagon answers with one rule you can hold in your head.

## 5.1 The rule

**Sequential bindings never shadow. Fresh-scope binders always may.**

That's it. Unpacking the two halves:

**Within a block, a name is taken.** Once you `let x = ...`, no later `let`, `var`, or `fun` in that scope may reuse `x` — not even in a nested block where JS would allow it. This is a hard compile error:

```
let process(order) =
  let total = subtotal(order)
  let total = total + tax(order)    // ERROR: `total` is already bound
  total
```

If you've written Rust you may *miss* this pattern (Rust encourages rebinding). Hexagon rejects it deliberately: when you read `total` on line 10 of a function, it means the same thing it meant on line 2. No mental version-tracking of which `total` is live. The fix is honest names:

```
let process(order) =
  let subtotal = subtotal(order)
  let total = subtotal + tax(order)
  total
```

**But binders that open a fresh scope shadow freely.** Function parameters, loop variables, and `match`-arm bindings (Chapter 7) are all allowed to reuse any name in scope:

```
let x = 99

let addOne(x) = x + 1    // fine: parameter x shadows the outer x

for x in 1..3           // fine: loop variable x shadows too
  print(show(x))        // 1, 2, 3 – the loop's x
```

## 5.2 Why the split?

The two cases *feel* different when you read code, and the rule tracks the feeling.

A parameter or loop variable sits in a *header* — it visibly announces "inside this region, `x` means my thing." Your eye sees the boundary. Shadowing here is harmless and forbidding it would be miserable: you'd be renaming lambda parameters to dodge every name in the enclosing function.

A second `let x` mid-block has no such boundary. It silently changes what `x` means from one line to the next, in the middle of prose. That's the one that causes 2 a.m. bugs, and that's the one Hexagon bans.

Compare the TypeScript situation: `let` redeclaration in the *same* scope is an error, but a nested block can shadow anything, silently:



```
// TypeScript – legal, and a classic bug
let total = 0;
for (const item of items) {
  let total = item.price;    // oops – new variable, outer total never
                             // updates
}
return total;                // 0, forever
```

Hexagon makes that inner binding a compile error (it's a sequential `let/var`, and the name is taken), while still letting you name your loop variable whatever you want. One rule, both footguns covered.

## Chapter 6 — `var`: mutability with a fence around it

Interesting feature number three. Hexagon is an immutable-by-default language — but it is not a purity zealot. Sometimes the clearest way to write an algorithm is a mutable accumulator and a loop, and Hexagon says: fine, here's `var`.

### 6.1 The basics

```
let collatzSteps(n0) =
  var n = n0
  var steps = 0
  while n != 1
    if isEven(n) then n := Int.div(n, 2) else n := 3 * n + 1
    steps := steps + 1
  steps
```

`var` declares a mutable local; `:=` assigns to it. (Assignment gets its own operator so that `=` can stay pure — you cannot accidentally assign in a condition, because `:=` isn't valid there and `==` is the only equality.)

This compiles to exactly the `let`-and-loop JavaScript you'd write yourself:

```
const collatzSteps = (n0) => {
  let n = n0;
  let steps = 0;
  while (n !== 1) {
    if (isEven(n)) { n = Int.div(n, 2); } else { n = 3 * n + 1; }
    // Int.div is one of those small helpers – it carries the divide-by-
    // zero check
    steps = steps + 1;
  }
  return steps;
};
```

### 6.2 The fence

Here's what makes Hexagon's **var** different from TypeScript's **let**: **a var cannot escape the function it was born in.**

- A **var** is local to a function body. There are no module-level **vars**, no mutable globals.
- A lambda **cannot capture a var** from an enclosing scope. This is a compile error:

```
let makeCounter() =
  var count = 0
  () => count := count + 1    // ERROR: lambda cannot capture the `var`
count
```

In JavaScript, that closure-over-mutable pattern is idiomatic — and it's also the source of every "why is my callback seeing a stale value" bug, every loop-variable-capture surprise, every action-at-a-distance state mutation. Hexagon draws the line differently: mutation is a *local implementation detail*. Inside a function, mutate freely; from the outside, every Hexagon function looks pure. If a value crosses a function boundary, it's immutable data.

(Loop bodies, **if** blocks, and **match** arms are *blocks*, not lambdas — so the loop in **collatzSteps** touching **n** is fine. The fence is at lambda boundaries specifically, which is exactly where the bugs live.)

### 6.3 What this buys you

You get the pleasant 90% of mutability — accumulators, counters, worklist loops, imperative algorithms transcribed straight from the textbook — while the compiler guarantees the dangerous 10% (shared mutable state, aliased mutation, closure capture) is unrepresentable. It's **const**-by-default taken seriously: not a lint rule, a type system fact.

## Chapter 7 — Unions and **match**: the feature TypeScript taught you to want

You already use discriminated unions in TypeScript. You already know they're the best part of the language. Hexagon's version is the same idea with the ceremony removed and the safety made mandatory.

### 7.1 Side by side

TypeScript:

```
// TypeScript
type Shape =
  | { tag: "circle"; radius: number }
  | { tag: "rect"; width: number; height: number }
  | { tag: "point" };

function area(s: Shape): number {
  switch (s.tag) {
    case "circle": return 3.14 * s.radius * s.radius;
    case "rect":   return s.width * s.height;
    case "point":  return 0;
```

```

    default: {
      const _exhaustive: never = s; // the ritual
      throw new Error("unreachable");
    }
  }
}

```

Hexagon:

```

union Shape =
| Circle(radius: Float)
| Rect(width: Float, height: Float)
| Point

let area(s) =
  match s
    Circle(r) => 3.14 * r * r
    Rect(w, h) => w * h
    Point => 0.0

```

Things to notice:

- **No annotation on `s`** — matching against `Circle` tells the compiler `s : Shape`. Inference again.
- **Constructors are functions:** `Circle(2.0)` builds a value; you can even pass the constructor around, as in `map(radii, Circle)`.
- **Patterns bind:** `Circle(r) =>` extracts the radius into `r` right in the arm. No `s.radius` re-narrowing dance.
- **match is an expression** — it has a value, so it slots into a function body or a `let` directly.

## 7.2 Exhaustiveness is not a ritual, it's the law

Delete the `Point` arm and Hexagon says:

```
error: match is missing cases: Point
```

Not a lint warning, not an opt-in `never` trick you have to remember — a hard compile error, always. Add a fourth constructor to `Shape` next year, and every `match` in the codebase that doesn't handle it fails to compile, with the missing constructor named. This is the refactoring superpower that TS's exhaustiveness-by-convention approximates; Hexagon just guarantees it. (Unreachable arms are errors too — a dead `case` is always a bug or a leftover.)

There's also no way *around* `match`: you cannot dot into a union value (`s.radius` on a `Shape` is a compile error saying "union values are inspected with `match`"). The type system never has to trust that you checked the tag, because checking the tag is the only door.

## 7.3 `Option` instead of `undefined`

The prelude ships two unions you'll use constantly:

```
union Option(a) = Some(value: a) | None
```

— plus a two-constructor `Result` union (success case first) for operations that fail with information attached.

Where TS APIs return `T | undefined` and hope you remember the `if`, Hexagon returns `Option(a)` — and since `match` is the only way in, forgetting to handle `None` is a compile error, not a runtime `undefined is not a function`. (At the JS boundary, nullable values are converted explicitly; `undefined` does not leak into the language.)

## 7.4 And the emitted JS?

Exactly what you'd hand-write:

```
// Circle(2.0) emits:
{ tag: "Circle", radius: 2.0 }

// the match emits:
switch (s.tag) {
  case "Circle": { const r = s.radius; return 3.14 * r * r; }
  case "Rect":   return s.width * s.height;
  case "Point":  return 0.0;
}
```

And the `.d.ts` is the discriminated union a TS author would write by hand:

```
type Shape =
  | { tag: "Circle"; radius: number }
  | { tag: "Rect"; width: number; height: number }
  | { tag: "Point" };
```

Your Hexagon modules are first-class citizens of a TypeScript codebase. (One bonus: a union whose constructors all carry no data — `union Color = Red | Green | Blue` — compiles to bare string literals, `"Red"`, exactly like the string-literal-union idiom you already use.)

---

## Chapter 8 — Records and tuples, briefly

You met records in Chapter 2. The short version:

```
{x: 1.0, y: 2.0}           // a record; compiles to the same object literal
let p2 = {...p, x: 3.0}    // non-destructive update; compiles to {...p, x:
3.0}
```

```
p.x                // field access

(1, "a")           // a tuple; compiles to [1, "a"]
let (lo, hi) = bounds // destructuring
```

Records are immutable plain objects; update is spread-copy, and the spread syntax emits *itself* — no lies in the output. One safety upgrade over JS spread: `{...p, z: 3.0}` where `p` has no field `z` is a compile error ("record update cannot add fields") rather than a silent widening. Updates update; they don't quietly grow new fields from a typo.

When you want a *named* type, `record Point = {x: Float, y: Float}` gives you a nominal wrapper: `Point` and a structurally identical anonymous record are different types to the compiler (goodbye, "accidentally passed a `UserId` where an `OrderId` was expected" — the structural-typing hole TS brands can only paper over). At runtime a `Point` is just the plain object; the nominal wall exists only at compile time, which is the only place it needs to exist.

## Chapter 9 — Small sharp edges (that are there on purpose)

A grab-bag of decisions you'll bump into early. Each one is a deliberate trade.

**No warnings.** Hexagon has exactly two outcomes: compiles, or hard error. There is no warning tier to accumulate, silence, or argue about in code review.

**You can't drop values on the floor.** In JS, `validate(x);` on its own line silently discards the result — even if the result was the error you needed. In Hexagon, a non-`Unit` value in statement position is a compile error. If discarding is genuinely what you want, say so: `ignore(validate(x))` — or, since `ignore` is an ordinary function, the preferred pipe spelling: `validate(x) |> ignore`. That form reads in order of events — do the thing, then discard the result — and keeps the interesting call at the front of the line. Either way, intent becomes visible; the "computed the answer and forgot to use it" bug becomes uncompileable.

**No `break`, no `continue` (v1).** Following F#'s precedent. Early exits are expressed by putting the condition in the `while` head, or restructuring into a function and returning. (A deeper investigation of `break/continue` is on the design backlog — this is a "not yet, prove the need" deferral, not a religious ban.)

**No C-style `for(;;)`.** Counting is `for i in 1..n` — inclusive on both ends, so that's  $n$  iterations, and it compiles to the classic `for (let i = 1; i <= n; i++)` counting loop. Irregular stepping is `while + var`.

**Exceptions exist, and they're real JS errors.** `throw(ParseError(3, "unexpected token"))`, and `try/catch` is an expression whose catch arms look exactly like `match` arms. Hexagon exceptions are genuine JS `Error` objects (branded so `catch` can distinguish them from foreign ones), so stack traces work and JS callers see nothing exotic. For expected failures, `Result` is the idiomatic tool; exceptions are for the exceptional.

**No operator overloading, ever.** `+` means numeric addition, `++` means concatenation, permanently, in every codebase you'll ever read. The set of operators is closed and each one means one thing.

## Chapter 10 — The JavaScript you get

A closing gallery, because "compiles to readable JavaScript" is a claim best proven by looking. Left: Hexagon. Right: emitted JS.

| Hexagon                                    | Emitted JavaScript                           |
|--------------------------------------------|----------------------------------------------|
| <code>let x = 5</code>                     | <code>const x = 5;</code>                    |
| <code>var n = 0 ... n := n + 1</code>      | <code>let n = 0; ... n = n + 1;</code>       |
| <code>let f(a, b) = a + b</code>           | <code>const f = (a, b) =&gt; a + b;</code>   |
| <code>fun fib(n) = ... (recursive)</code>  | <code>function fib(n) { ... }</code>         |
| <code>for i in 1..n</code>                 | <code>for (let i = 1; i &lt;= n; i++)</code> |
| <code>for x in xs</code>                   | <code>for (const x of xs)</code>             |
| <code>{x: 1.0, y: 2.0}</code>              | <code>{x: 1.0, y: 2.0}</code>                |
| <code>{...p, x: 3.0}</code>                | <code>{...p, x: 3.0}</code>                  |
| <code>(1, "a")</code>                      | <code>[1, "a"]</code>                        |
| <code>Circle(2.0)</code>                   | <code>{tag: "Circle", radius: 2.0}</code>    |
| <code>match s ...</code>                   | <code>switch (s.tag) { ... }</code>          |
| <code>a and b, a or b, not a</code>        | <code>a &amp;&amp; b, a    b, !a</code>      |
| <code>xs  &gt; map(f)  &gt; take(3)</code> | <code>take(map(xs, f), 3)</code>             |

The pattern: Hexagon's abstractions are *compile-time* abstractions. Types, inference, exhaustiveness, the immutability rules, the `var` fence, pipes — all of it does its work during compilation and then erases. What reaches the runtime is the JavaScript a careful senior engineer would have written, plus a `.d.ts` that tells TypeScript the truth.

## What this book left out

Plenty — this is a preview, not a manual. Among the things not covered: the constraint system that powers `Num`, `Eq`, `Ord`, and `Show` (Hexagon's principled answer to ad-hoc polymorphism — think Rust traits, no inheritance); lazy sequences (`Seq`) and the iteration protocol; the full story on exceptions and the JS interop boundary; string details; and everything still being designed — deep pattern matching, modules, async, and the standard library.

The design is documented as it happens, with every decision — and every *rejected* alternative — written down with its reasoning. If this tour made you curious, that's the right reaction: Hexagon is being built for exactly the developer who read this far.

*Hexagon is in active design. Syntax and semantics shown here reflect the specification as of mid-2026 and may evolve before release.*